

Aarhus University

Project number - 2026F25

Multiplayer

Students:

Thea Teresa Lindgaard (202204350)

Niels Jakob Østerberg (202105879)

Kasper Stecher (202100451)

Kasper Bentsen (202103249)

Counsellor:

Lukas Esterle

Aarhus, May 29th 2026

Contents

1. Introduction	1
1.1. Requirements	1
2. Design	2
2.1. Architecture	2
2.1.1. C4	4
2.1.2. Design Patterns	4
2.1.2.1. Client-Server pattern	4
2.1.2.2. Singleton pattern	5
2.1.2.3. Observer pattern	5
2.2. Classes	6
2.2.1. PersistentSingleton	6
2.2.2. AuthenticationServiceWrapper	6
2.2.3. LobbyServiceWrapper	7
2.2.4. RelayServiceWrapper	7
2.2.5. SceneLoader	7
2.2.6. MultiplayerManager	7
2.3. Synchronization Strategy	8
2.4. Handling Assymetric Gameplay	8
2.5. Connection Flow	9
2.6. Testing Strategy	12
3. Implementation	13
3.1. PersistentSingleton<T>	13
3.2. AuthenticationServiceWrapper	14
3.3. LobbyServiceWrapper	15
3.4. RelayServiceWrapper	16
3.5. SceneLoader	17
3.6. MultiplayerManager	17
4. Testing Execution	19
4.1. Functional Testing	19
4.2. Integration Testing	19
4.3. Manual Playtesting	20
4.4. Network Condition Simulation	20
4.5. Observations	22
4.6. Summary	23
5. Conclusion of the Multiplayer System	24
Bibliography	25

1. Introduction

Author: Kasper Bentsen

The purpose of the multiplayer system (**MPS**), is to facilitate online connectivity between players and allow them to cooperate and solve the game challenges.

The multiplayer system pertains to players connecting to each other via player hosted peer-to-peer (**P2P**), implemented through a host-client architecture using relayed network transport, and forming parties and lobbies, to overcome the different game levels.

The functionality is implemented using *Unity Netcode for GameObjects* combined with *Unity Relay services*, allowing peer hosted multiplayer gameplay without requiring dedicated server infrastructure. The architecture follows a *host-client model*, where one player acts as the host and maintains the authoritative game state.

1.1. Requirements

The requirements, as outlined in **Appendix C: Requirements Specification**, are:

Functional

- 1) Synchronization of player characters (**E2-US1**)
- 2) Synchronization of game objects (**E2-US2**) + (**E2-US3**)
- 3) Visual and/or auditory communication (**E2-US5**)
- 4) Voice communication (**E2-US6**)
- 5) The ability to create and join lobbies (**E4-US1 & E4-US2**)
- 6) Automatic reconnect on disconnects (**E4-US4**)
- 7) Separate handling of Client and Host leaving a game (**E4-US5**) + (**E4-US6**)
- 8) Host reloading game levels (**E6-US5**)

Non-functional

- 1) Less than 500ms latency (**Reliability - Should**)
- 2) Automatic reconnect attempts for 30s following connection loss (**Reliability - Could**)
- 3) Redirect to main menu after 30s of reconnect attempts (**Reliability - Should**)
- 4) Network latency should not introduce more than 150ms input delay (**Performance - Should**)
- 5) Scene transition time should not exceed 5 seconds (**Performance - Should**)
- 6) Server-side validation of client inputs (**Security - Could**)
- 7) Server-side Anti cheat validation (**Security - Could**)

These requirements serve as a guideline to ensure a smooth and enjoyable player experience, as well as basic functionality for the system to work at all.

Most of these have been implemented, but some were cut due to time constraints, such as anti cheat, server-side client input validation and voice communication.

2. Design

In **Appendix I.1.1: Multiplayer Analysis**, multiple forms of multiplayer architecture were analyzed, and a **host-client** architecture were found to be best fit for the Parallax project. Its simpler implementation, reduced cost and suitability for smaller lobby sizes, perfectly fit the scope of the project.

As such, the **MPS** is designed to match the classic **Host-Client** architecture. Here, one player acts as a host and the other as a client. The host runs the authoritative game state and the client sends input to the host, who synchronizes objects between the two players.

This follows the standard *Unity Netcode for Gameobjects* architecture, and is implemented via the *Netcode for Gameobjects* and *Unity Multiplayer Services* packages.

2.1. Architecture

In **Figure 1**, a brief overview of the host-client model is displayed.

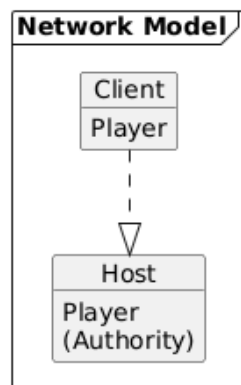


Figure 1: Overview of the Host-Client network model

The client connects to the host through relay servers provided by Unity.

To create and join lobbies, Unity's lobby and relay systems has some required steps, depicted in **Figure 2**.

The players are initially authenticated through Unity Authentication, where they are signed in anonymously and initialized.

Once signed in, the player can choose to create or join a lobby, thus acting as the host or client respectively.

As the **host**, the player is allocated a relay, which has a relay join-code. A lobby is created and the relay join code is assigned to the lobby. The NetworkManager is then updated with data from the relay allocation, and the host is started.

As the **client**, the player can either quick-join or join via code. The flow is the same and the difference is handled internally in the MPS. If the player chooses to quick-join, they are automatically assigned a lobby, and the relay join-code is extracted from there. If joining via code, the lobby is specifically

assigned. The NetworkManager is updated with the data from the relay allocation, and the client is started.

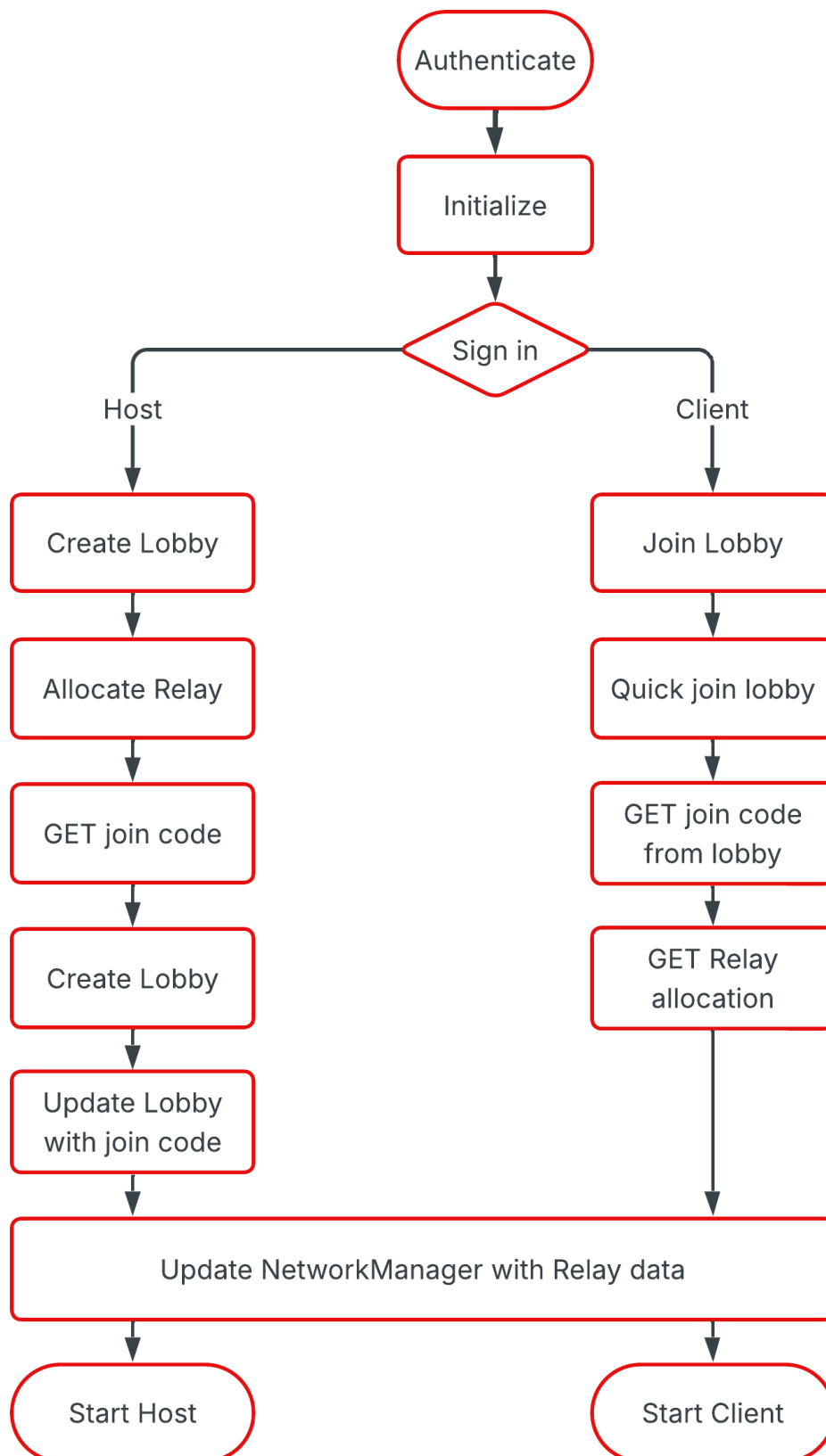


Figure 2: Lobby and Relay connection flow

2.1.1. C4

The diagram in **Figure 3** depicts a C4 Level 1 System Context Diagram illustrating the multiplayer networking architecture of Parallax using *Unity Netcode for GameObjects*, *Unity Relay*, and *Unity Lobby*.

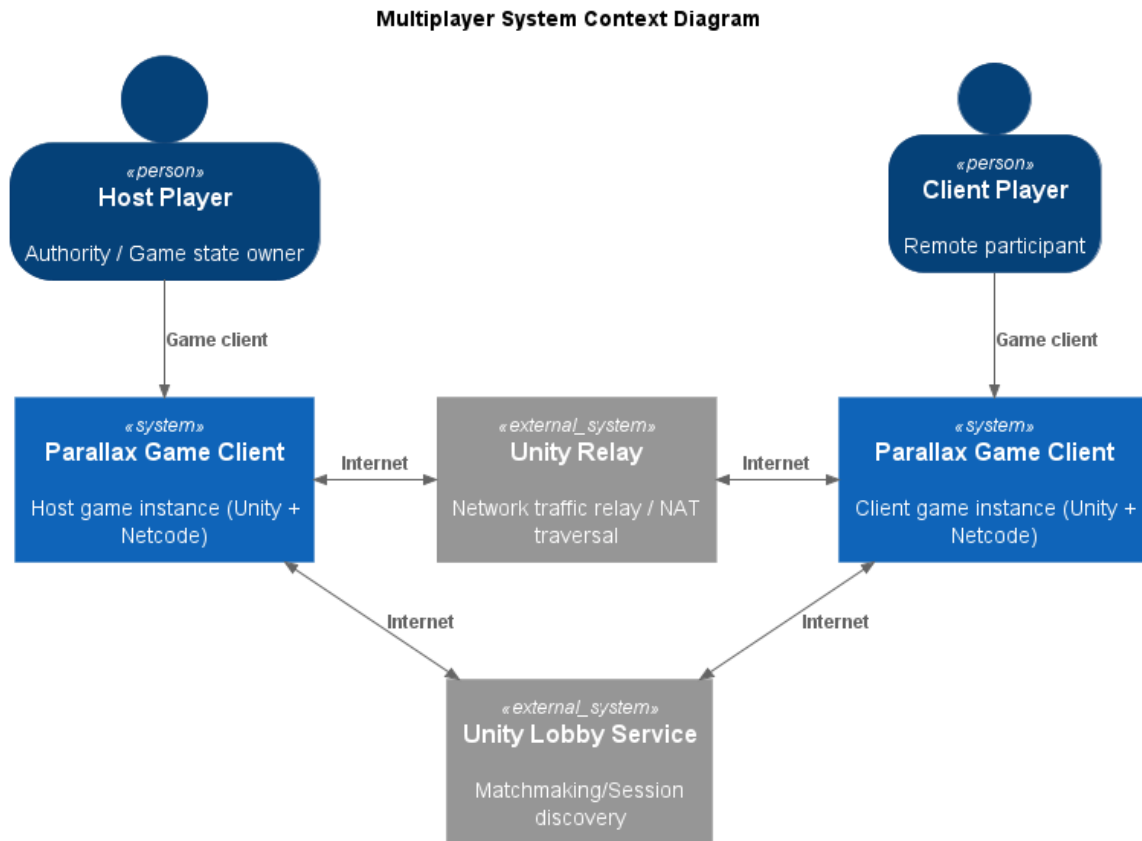


Figure 3: System Context C4 diagram of the multiplayer system

Parallax uses a host-authoritative model, where either participant may assume the role of host, implemented with *Unity Netcode for GameObjects*. The game instance running as Host owns the authoritative game state. Network traffic mediation and NAT traversal is handled via *Unity Relay Service*, while matchmaking and session discovery is managed by *Unity Lobby Service*.

2.1.2. Design Patterns

Multiple established design patterns were utilized in the development of the **MPS**. Specifically the *Client-Server pattern*[1], [2], *Singleton pattern*[3] and the *Observer pattern*[4].

2.1.2.1. Client-Server pattern

The *client server pattern* is a network communication pattern, that specifies a host to act as a server and maintain the authoritative state, to which clients can connect and send requests. The host then distributes updates to the connected clients.

In the **MPS**, the architecture follows a host-client model, where one player acts as the host, while the other joins as a client. The host is responsible for synchronizing state between the connected players.

Clients register inputs and send them to the host via *Unity Transport*. The host processes the input and distributes the resulting state changes to the client. This is how game world consistency is ensured.

The pattern simplifies synchronization and reduces the risk of conflicting game states.

2.1.2.2. Singleton pattern

Singleton is a creational design pattern that helps ensure that a class has only one instance, while providing a global access point to this instance.

Unity's built-in **NetworkManager** follows a Singleton-like access pattern. Other classes can access it via the **NetworkManager.Singleton** access specifier.

The base responsibilities of the **NetworkManager** are:

- Launching host or client sessions
- Managing player connections
- Handling disconnections
- Initializing network objects

The *singleton pattern* simplifies access to the networking functionalities from other game systems.

2.1.2.3. Observer pattern

The *Observer pattern* allows classes to subscribe to events generated by other classes. When the state of the observed class changes, all subscribers are notified automatically and acts accordingly.

The multiplayer system relies on event-driven communication between components. Network events such as player connections, disconnections, and object state changes trigger updates in other parts of the system.

In the **MPS**, events are primarily used for debug logging, but the `OnClientDisconnected` event is responsible for handling disconnects correctly, based on the current state of the game, and if the disconnected client were hosting the server or not.

When a player joins the session, an event is triggered in the **NetworkManager**, that spawns the corresponding player object and updates the game state. Similarly, when network variables change, subscribed components receive notifications and update their local state.

This pattern helps decouple networking logic from gameplay systems.

2.2. Classes

The **MPS** consists of multiple classes that each encapsulate a specific area of responsibility. This separation improves maintainability and reduces coupling between systems.

The classes are primarily responsible for authentication, lobby management, relay configuration, scene synchronization and overall multiplayer orchestration.

A class diagram illustrating the relationships between these classes is shown in **Figure 4**.

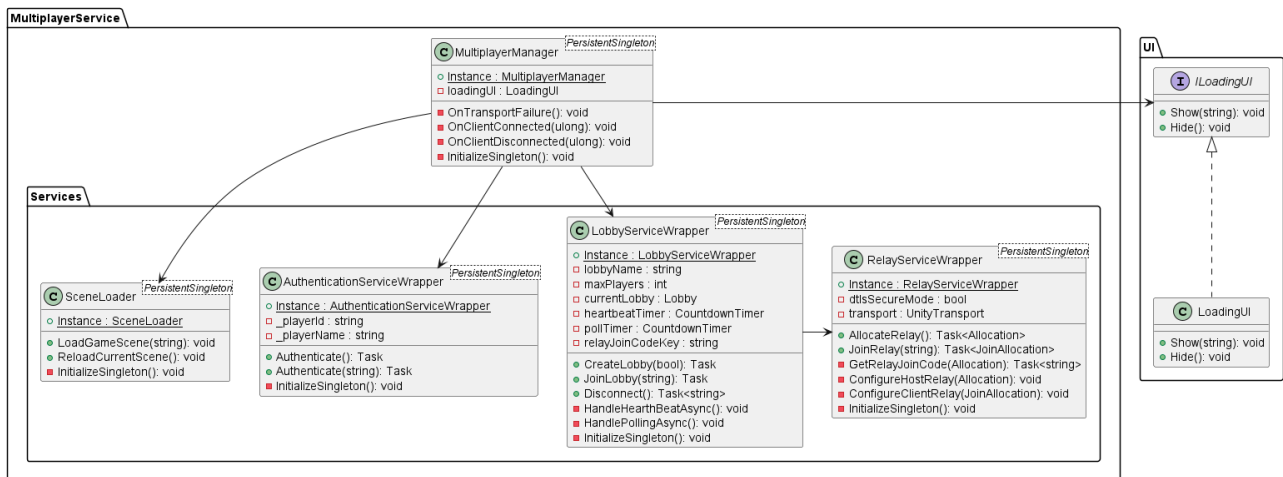


Figure 4: Class diagram of the multiplayer system

2.2.1. PersistentSingleton

The *PersistentSingleton* base class, is implemented as an abstract template class, that instantiates its implementor as a singleton game object, with **DontDestroyOnLoad()**.

The five classes that implement *PersistentSingleton* are

- *MultiplayerManager*
- *AuthenticationServiceWrapper*
- *LobbyServiceWrapper*
- *RelayServiceWrapper*
- *SceneLoader*

2.2.2. AuthenticationServiceWrapper

The *AuthenticationServiceWrapper* class is responsible for handling player authentication and identity management. It ensures that each player is assigned a unique identity before interacting with other multiplayer services.

The class provides an abstraction layer between the multiplayer system and the external Unity authentication services, allowing other components to request authenticated player information without depending directly on the underlying authentication provider.

This helps to loosen coupling and simplifies future replacement or extension of the authentication system.

2.2.3. LobbyServiceWrapper

The *LobbyServiceWrapper* class is responsible for managing multiplayer session creation, discovery and lifecycle.

Its responsibilities include creating and joining lobbies, maintaining active sessions and handling player departures.

The class acts as an abstraction layer between the multiplayer system and Unity Lobby Services.

It also coordinates with the relay system to associate connection information with active lobbies, enabling players to connect to the correct game session.

By encapsulating lobby-related responsibilities in a dedicated class, session management is isolated from gameplay and UI systems.

2.2.4. RelayServiceWrapper

The *RelayServiceWrapper* class is responsible for managing network transport mediation between connected players.

Its primary purpose is to establish and configure relay-based connections between host and client instances.

The class abstracts communication with Unity Relay Services and hides relay-specific connection details from the rest of the system.

This separation simplifies the networking architecture and reduces dependencies between systems.

2.2.5. SceneLoader

The *SceneLoader* class is responsible for handling synchronized scene transitions in multiplayer sessions.

It ensures that connected players remain in the same game state by coordinating scene loading across the network.

This class abstracts scene synchronization logic from other systems and provides a centralized interface for changing or reloading scenes.

This improves maintainability and reduces duplication of scene-loading logic.

2.2.6. MultiplayerManager

The *MultiplayerManager* class acts as the central coordinator of the Multiplayer System. It is responsible for orchestrating the interaction between the authentication, lobby, relay and scene management systems.

The class serves as the main entry point for multiplayer-related actions and acts as the bridge between UI systems and backend multiplayer services. By centralizing orchestration logic in a single class, the system maintains clear separation of responsibilities while simplifying communication between subsystems.

2.3. Synchronization Strategy

The Multiplayer System uses a *host-authoritative synchronization strategy*.

In this model, the host maintains the authoritative game state, while connected clients submit input and receive synchronized updates.

This strategy was chosen to ensure consistency between players and reduce the likelihood of conflicting game states.

Synchronization is primarily handled through *Unity Netcode for GameObjects*.

Player movement, object positions and interactable object states are synchronized using Unity's built-in networking components such as *NetworkObject*, *NetworkVariable*, *NetworkTransform* and *RPCs*.

The system distinguishes between different forms of synchronization depending on the nature of the data:

- **Continuous synchronization** is used for frequently changing values such as player movement and object transforms.
- **State synchronization** is used for persistent object states such as whether an item has been collected or a door has been unlocked.

While the host maintains the overall authoritative game state, player movement is based on an owner-authoritative model.

By combining these strategies, the system reduces unnecessary network traffic while maintaining a consistent shared game state.

2.4. Handling Assymmetric Gameplay

Parallax is designed around asymmetric cooperative gameplay, where players experience different perspectives of the same environment.

Although players perceive the game world differently, the underlying game state remains shared and synchronized across all connected players.

The **MPS** therefore synchronizes gameplay-critical objects and interactions identically between players using the standard synchronization mechanisms provided by Unity Netcode for GameObjects.

Asymmetry is instead handled locally by each game client through role-based rendering and gameplay logic.

Depending on the role assigned to the local player, the client may render different visual elements, enable or disable specific interactions, present unique information or clues or alter how certain objects are displayed.

For example, one player may perceive an object differently through alternative models, textures or visual effects, while the synchronized underlying object state remains the same across both clients.

This approach simplifies the networking architecture by maintaining a single authoritative shared game state while still enabling asymmetric gameplay experiences.

The host remains authoritative over all synchronized game logic, while each local client is responsible for presenting the appropriate perspective based on the assigned role.

2.5. Connection Flow

The **MPS** follows a structured connection flow to establish multiplayer sessions.

This flow is demonstrated in “**Figure 2: Lobby and Relay connection flow**” as a flow diagram, and with implementation details in “**Figure 6: Lobby and Relay connection flow**” as a sequence diagram.

Before a player can host or join a game session, they are first authenticated through *Unity Authentication Services*. For Parallax, the players are authenticated anonymously, since actual identity providers are not within the scope of the project. Each player is assigned a standard anonymous identifier.

In a full version of the game, authentication could be setup with different identity providers, such as Google, Facebook, Steam, Apple or even a simple username and password implementation.

Once authenticated, the player may either create or join a lobby.

If the player creates a lobby:

- 1) A Relay allocation is created.
- 2) A Relay join code is generated.
- 3) A Lobby is created.
- 4) The Relay join code is stored in the lobby metadata.
- 5) The game instance starts in Host mode.

If the player joins a lobby:

- 1) The player joins through quick join or lobby code.
- 2) The Relay join code is retrieved from the lobby metadata.
- 3) The Relay allocation is joined.
- 4) The game instance starts in Client mode.

Once connected, gameplay synchronization is handled through *Unity Netcode for GameObjects* using *Unity Relay* as the transport intermediary.

This flow ensures a structured and reliable process for establishing multiplayer sessions.

To communicate to the player that lobby creation/joining is in progress, a free spinner asset[5] from the Unity asset store is utilized in conjunction with a status text. Example shown in **Figure 5**

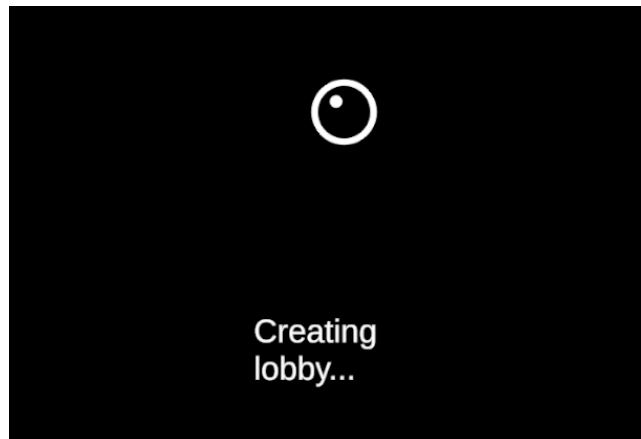


Figure 5: Spinner and loading text (Host example)

The spinner and text element are activated/deactivated directly from within the *MultiplayerManager* class, at the appropriate times.

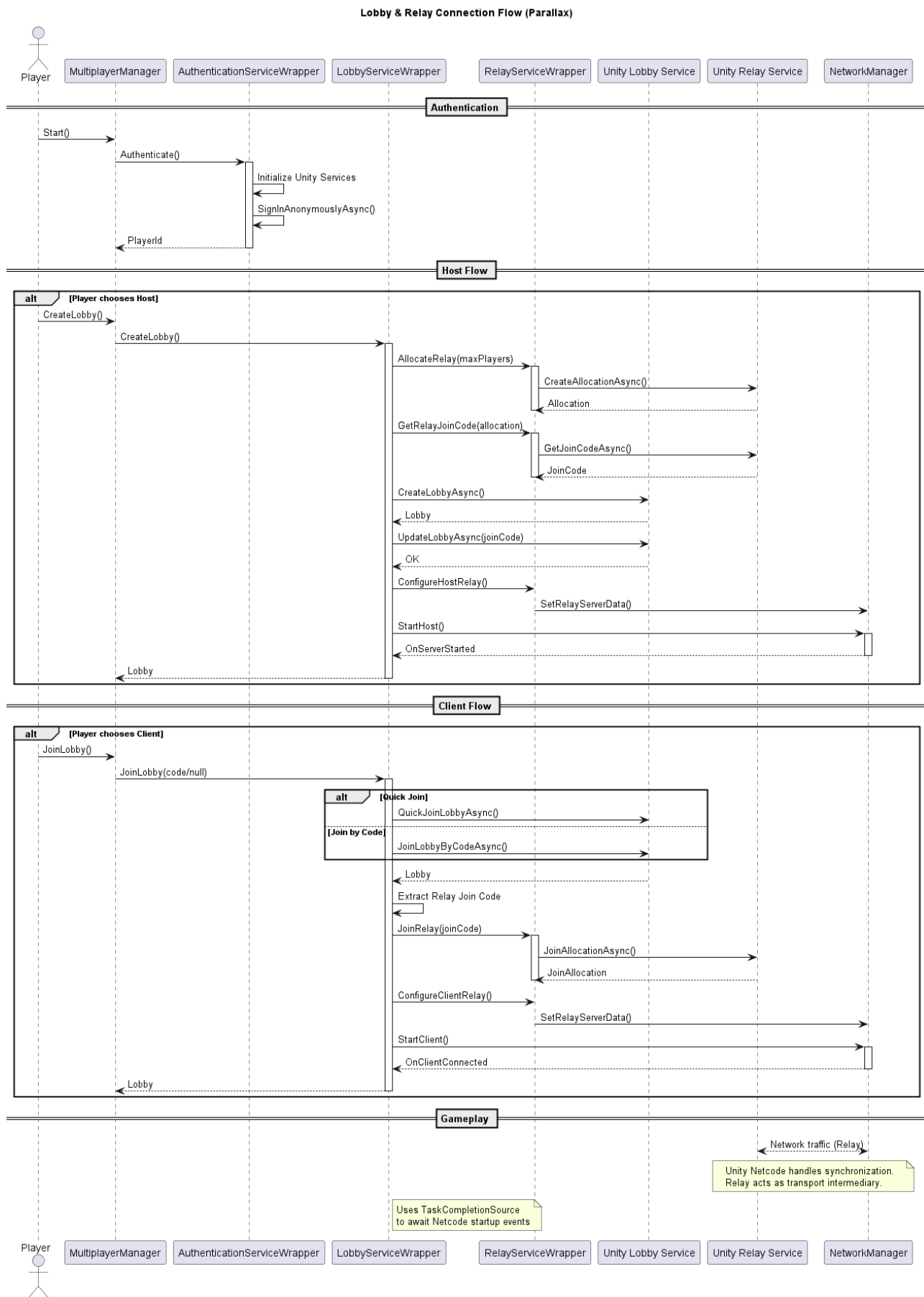


Figure 6: Lobby and Relay connection SD

2.6. Testing Strategy

The Multiplayer System was tested through a combination of *functional testing*, *integration testing* and *manual playtesting*.

Functional testing was used to verify individual features and requirements, such as:

- successful authentication
- lobby creation and joining
- player synchronization
- object synchronization
- scene transitions
- disconnect handling

Integration testing was used to verify communication between systems, particularly between:

- Unity Authentication Services
- Unity Lobby Services
- Unity Relay Services
- Unity Netcode for GameObjects

This ensured that the complete connection flow functioned as intended.

Manual playtesting was used to evaluate real gameplay scenarios and identify issues related to latency, synchronization and asymmetric interactions.

Specific test scenarios included:

- Host disconnecting during gameplay
- Client disconnecting during gameplay
- reconnect attempts after connection loss
- scene reloads during active sessions
- synchronization of interactable objects
- synchronization of role-specific gameplay elements

Testing was performed across separate devices and networks to validate Relay-based connectivity.

This multi-layered testing strategy helped identify implementation issues early and improve the reliability of the Multiplayer System.

3. Implementation

The Multiplayer System is implemented as a set of modular classes, each encapsulating a specific area of functionality.

The implementation builds upon Unity's built-in multiplayer frameworks, including *Unity Netcode for GameObjects*, *Unity Lobby Services* and *Unity Relay Services*.

The following sections describe the implementation details of each core class.

3.1. PersistentSingleton<T>

The **PersistentSingleton<T>** class is implemented as a generic abstract class inheriting from **MonoBehaviour**.

It provides a reusable Singleton implementation for systems that must persist throughout the game session.

A static **Instance** field stores the globally accessible instance of the class.

In the **Awake()** method, the class checks whether another instance already exists. If so, the duplicate object is destroyed to enforce the Singleton pattern.

```
protected virtual void Awake()
{
    if (Instance != null && Instance != this)
    {
        Destroy(gameObject);
        return;
    }

    Instance = this as T;

    if (dontDestroyOnLoad && transform.parent == null)
    {
        DontDestroyOnLoad(gameObject);
    }
}
```

If no instance exists, the current object is assigned as the active instance.

Optionally, the object can be marked with **DontDestroyOnLoad()**, ensuring it persists between scene transitions.

This implementation reduces duplicate Singleton logic across multiple manager classes.

3.2. AuthenticationServiceWrapper

The **AuthenticationServiceWrapper** class is implemented using *Unity Authentication Services* and *Unity Core Services*.

Authentication is performed asynchronously using *Task*-based methods.

```
public async Task<string> Authenticate()
{
    string playerId = "";
    playerId = await Authenticate("Player" + Random.Range(0, 1000));
    return playerId;
}

public async Task<string> Authenticate(string playerName)
{
    if (UnityServices.State == ServicesInitializationState.Uninitialized)
    {
        InitializationOptions options = new();
        options.SetProfile(playerName);
        await UnityServices.InitializeAsync(options);
    }

    AuthenticationService.Instance.SignedIn += () =>
    {
        Debug.Log("Signed in as " + AuthenticationService.Instance.PlayerId);
    };

    if (!AuthenticationService.Instance.IsSignedIn)
    {
        await AuthenticationService.Instance.SignInAnonymouslyAsync();

        _playerId = AuthenticationService.Instance.PlayerId;
        _playerName = playerName;
    }

    return _playerId;
}
```

Before authentication can occur, Unity Services must be initialized. This is done through **UnityServices.InitializeAsync()**.

A player profile is assigned through **InitializationOptions**, allowing multiple local instances to run with separate authentication contexts.

The player is then signed in anonymously using the Authentication service's **SignInAnonymouslyAsync()**.

Upon successful authentication, Unity generates a unique **Player ID**, which is stored locally along with the player name.

The class also subscribes to the **SignedIn** event for debugging and verification of successful authentication.

This implementation ensures that all players are authenticated before accessing multiplayer functionality.

3.3. LobbyServiceWrapper

The **LobbyServiceWrapper** class is implemented using *Unity Lobby Services*.

The class manages lobby creation, joining, polling and disconnection using asynchronous methods.

When creating a lobby, the system first requests a relay allocation through the **RelayServiceWrapper**.

The resulting relay join code is stored as lobby metadata, allowing joining clients to retrieve it.

```
Allocation allocation = await RelayServiceWrapper.Instance.AllocateRelay(maxPlayers);  
string relayJoinCode = await RelayServiceWrapper.Instance.GetRelayJoinCode(allocation);
```

```
currentLobby = await LobbyService.Instance.CreateLobbyAsync(lobbyName, maxPlayers);
```

```
await LobbyService.Instance.UpdateLobbyAsync(currentLobby.Id,  
    new UpdateLobbyOptions  
    {  
        Data = new Dictionary<string, DataObject>  
        {  
            { k_keyJoinCode, new DataObject(DataObject.VisibilityOptions.Member,  
relayJoinCode) }  
        }  
    });
```

Lobby creation is performed using **CreateLobbyAsync()**.

Players can join lobbies using either:

- **JoinLobbyByCodeAsync()**
- **QuickJoinLobbyAsync()**

To keep lobbies alive, the host periodically sends heartbeat pings using **SendHeartbeatPingAsync()**.

The lobby state is periodically updated using **GetLobbyAsync()**.

These periodic actions are managed using **CountdownTimer** instances.

```
heartbeatTimer.OnTimerStop += () =>
{
    _ = HandleHeartbeatAsync();
    heartbeatTimer.Start();
};
```

```
pollTimer.OnTimerStop += () =>
{
    _ = HandlePollingAsync();
    pollTimer.Start();
};
```

When a player disconnects, the class handles cleanup by either:

- deleting the lobby if the host leaves
- removing the player if a client leaves

After lobby setup is complete, the class starts the host or client through **StartHost()** or **StartClient()** on the **NetworkManager**.

```
NetworkManager.Singleton.OnServerStarted += OnServerStartedHandler;
NetworkManager.Singleton.StartHost();
```

```
await tcs.Task;
```

or for client:

```
NetworkManager.Singleton.OnClientConnectedCallback += OnClientConnectedHandler;
NetworkManager.Singleton.StartClient();
```

```
await tcs.Task;
```

Task completion sources are used to await successful startup events before continuing execution.

This implementation ensures robust session handling and synchronization between players.

3.4. RelayServiceWrapper

The **RelayServiceWrapper** class is implemented using **Unity Relay Services**.

The class handles creation and joining of relay allocations.

Hosts request a relay allocation using **CreateAllocationAsync()**.

Once created, a relay join code is retrieved using **GetJoinCodeAsync()**.

Clients use this join code to join the allocation through **JoinAllocationAsync()**.

The returned allocation data is then used to configure the **UnityTransport** component attached to the **NetworkManager**.

```
NetworkManager.Singleton.GetComponent<UnityTransport>()
    .SetRelayServerData(new RelayServerData(
        allocation.RelayServer.IpV4,
        (ushort)allocation.RelayServer.Port,
        allocation.AllocationIdBytes,
        allocation.ConnectionData,
        allocation.ConnectionData,
        allocation.Key,
        dtlsSecureMode));
```

This is done through **SetRelayServerData()**.

Separate configuration methods exist for:

- host relay configuration
- client relay configuration

The class also supports secure DTLS transport mode through a configurable boolean setting. This is however not used in Parallax, as it was outside the scope of the project and the game is only played within the group.

This implementation abstracts low-level transport configuration from the rest of the multiplayer system.

3.5. SceneLoader

The **SceneLoader** class is implemented using Unity Netcode's synchronized scene management system.

Scene transitions are performed with **LoadScene()**, on the **SceneManager** in the **NetworkManager**.

This ensures that all connected clients transition scenes simultaneously.

The class provides methods for:

- loading a specified scene
- reloading the currently active scene

By centralizing scene-loading functionality, the implementation ensures consistency and simplifies scene transitions across the game.

3.6. MultiplayerManager

The **MultiplayerManager** class is implemented as the primary orchestration layer of the Multiplayer System.

At startup, the class registers Unity Netcode callbacks for:

- transport failures
- client connections

- client disconnections
- server startup

```
NetworkManager.Singleton.OnTransportFailure += OnTransportFailure;  
NetworkManager.Singleton.OnClientDisconnectCallback += OnClientDisconnected;  
NetworkManager.Singleton.OnClientConnectedCallback += OnClientConnected;
```

The class also initiates player authentication.

```
private async void Start()  
{  
    RegisterNetworkCallbacks();  
    await Authenticate();  
}
```

Lobby creation and joining requests are forwarded to the **LobbyServiceWrapper**.

```
currentLobby = await LobbyServiceWrapper.Instance.CreateLobby(isPrivate, maxPlayers);
```

```
SceneLoader.Instance.LoadGameScene(LobbySceneName);
```

Scene transitions are handled through the **SceneLoader**.

The class communicates with UI components such as **LoadingUI** to provide user feedback during asynchronous operations.

When players disconnect, callback handlers update the lobby state and remove disconnected players where necessary.

The class also handles transport failure scenarios and logs errors for debugging.

This implementation centralizes multiplayer orchestration and acts as the bridge between frontend and backend multiplayer systems.

4. Testing Execution

The Multiplayer System was tested through a series of structured test sessions conducted under both normal and degraded network conditions. Testing was performed using multiple physical machines on separate networks to simulate real-world player environments.

4.1. Functional Testing

Functional testing was carried out continuously during development to validate that core features behaved as expected in isolation. Each feature was tested directly within the Unity Editor and through standalone builds.

The following scenarios were executed:

- User authentication using Unity Authentication Services
- Lobby creation, joining, and leaving flows
- Host and client initialization
- Player spawning and ownership assignment
- Synchronization of player movement and transforms
- Synchronization of interactable objects across clients
- Scene transitions between lobby and gameplay
- Handling of client and host disconnections

Each test was repeated multiple times to ensure consistency, particularly after changes to networking logic or object synchronization.

4.2. Integration Testing

Integration testing focused on validating the full multiplayer connection pipeline across Unity services. These tests were performed using two separate clients, typically running on different machines, to replicate real multiplayer conditions.

The complete connection flow was tested as follows:

- 1) Player authentication via Unity Authentication Services
- 2) Lobby creation using Unity Lobby Services
- 3) Relay allocation and join code distribution via Unity Relay Services
- 4) Client connection using Unity Netcode for GameObjects

Special attention was given to verifying that data flowed correctly between services, and that failures in one service were handled gracefully by the system.

Test scenarios included:

- Clients joining lobbies via Relay join codes
- Transition from lobby scene to gameplay scene

- Late joining clients synchronizing correctly with the current game state
- Multiple reconnect attempts after connection loss
- Validation of network object ownership across clients

4.3. Manual Playtesting

Manual playtesting was conducted to evaluate the system under realistic gameplay conditions. These sessions involved actively playing the game with two participants, focusing on both gameplay experience and network stability.

Testing was performed across different devices and network setups, including local networks and external internet connections.

Key scenarios included:

- Host disconnecting during active gameplay
- Client disconnecting and attempting to reconnect
- Scene reloads while clients were connected
- Interaction with synchronized objects during latency
- Role-specific gameplay interactions under network delay

4.4. Network Condition Simulation

To further evaluate robustness, network conditions were artificially degraded using Unity's Network Simulator. Both clients were configured with a **"Home Broadband with Congested Network"** preset to simulate real-world latency, jitter, and packet loss, as demonstrated in **Figure 7**.

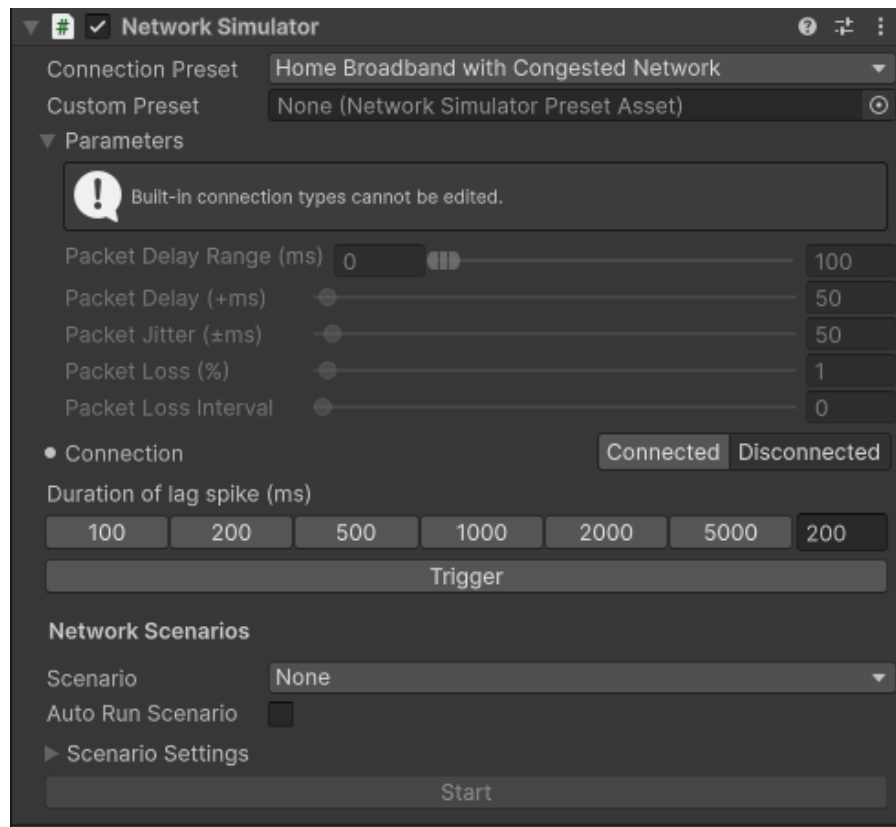


Figure 7: Unity Network Simulator configured with a congested broadband preset used during testing.

This allowed testing under conditions such as:

- Increased round-trip time (RTT)
- Packet delay and jitter
- Reduced bandwidth scenarios

During these tests, runtime network statistics were monitored in-game to observe:

- RTT to server
- Bytes sent and received per second
- Packet transmission rates
- RPC call frequency

This is demonstrated in **Figure 8**.

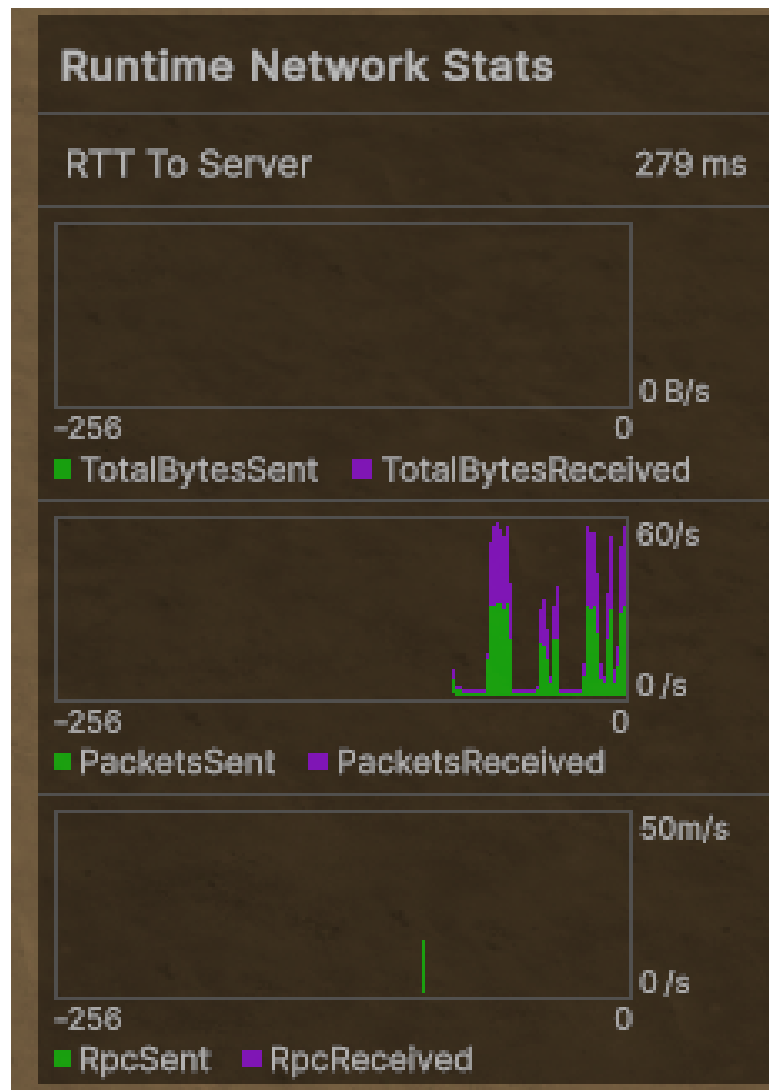


Figure 8: Runtime network statistics showing RTT, packet flow, and synchronization activity during simulated network conditions.

These metrics were used to identify synchronization issues, delayed updates, and potential bottlenecks in the networking layer.

4.5. Observations

Testing under simulated network constraints revealed several important behaviors:

- Increased latency resulted in noticeable delays in object synchronization and player movement
- Rapid state changes (e.g., picking up objects) were more prone to desynchronization under packet loss
- Scene transitions remained stable but required additional validation for edge cases such as mid-transition disconnects

Despite degraded conditions, the system maintained overall connectivity and recovered correctly from most simulated failures.

4.6. Summary

Testing was conducted iteratively throughout development, combining isolated feature validation, full system integration testing, and real-world gameplay sessions. By including both normal and simulated adverse network conditions, the Multiplayer System was evaluated for both correctness and robustness, ensuring reliable behavior across a variety of multiplayer scenarios.

5. Conclusion of the Multiplayer System

The development of the Multiplayer System resulted in a functional and robust solution for establishing and maintaining online co-operative gameplay sessions. The system successfully integrates Unity Authentication, Lobby, Relay, and Netcode for GameObjects into a unified workflow that enables players to connect, synchronize, and interact within a shared environment.

Through iterative implementation and testing, the system demonstrated reliable handling of core multiplayer functionality, including lobby creation and joining, player synchronization, scene transitions, and disconnection management. Integration between the different Unity services functioned as intended, supporting a complete end-to-end connection flow.

While the initial plan was to have a fully host-authoritative architecture, an owner-authoritative model was chosen for player movement. During the final weeks of the project, it was discovered that the initial model implemented was not host-authoritative and due to time constraints, and the fact that the game is not a competitive one, but a cooperative in nature, it was decided that the owner-authoritative model would suffice.

Testing under both normal and simulated adverse network conditions showed that the system maintains stable connectivity and acceptable performance, even in the presence of increased latency, jitter, and packet loss.

Additionally, the system supports the requirements imposed by the game's asymmetric design, ensuring consistent synchronization of role-specific gameplay elements across clients.

While minor synchronization delays and edge cases were observed under degraded network conditions, these did not compromise the overall functionality of the system. The Multiplayer System is therefore considered sufficiently robust within the scope of this project and provides a solid technical foundation for further development.

Bibliography

- [1] Unity, "Simple client and server." [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.transport@2.0/manual/client-server-simple.html>
- [2] G. for Geeks, "Client-Server Model." [Online]. Available: <https://www.geeksforgeeks.org/system-design/client-server-model/>
- [3] R. Guru, "Singleton." [Online]. Available: <https://refactoring.guru/design-patterns/singleton>
- [4] R. Guru, "Observer." [Online]. Available: <https://refactoring.guru/design-patterns/observer>
- [5] Hippo, "Simple Spinner." [Online]. Available: <https://assetstore.unity.com/packages/2d/gui/icons/simple-spinner-progress-indicators-for-ui-237500>